

CANDY DIALOGUE ENGINE

Starting Dialogues

1.0.1

The **Candy_Engine.gd** script contains the core engine code, which runs dialogues. Dialogues must be loaded into the **running_dialogue** dictionary in order to be played.

If you write dialogues in the Candy Dialogue Creator, it exports dialogue scripts to .txt files. These dialogue scripts can then be copied or loaded from these files.

A dialogue script is the name given to a collection of Conversations inside a same dictionary. One dialogue .txt file = one dialogue script.

If your game uses multiple instances of **Candy_Engine.gd**, you'll need to ensure that dialogues are loaded to the correct instance's **running_dialogue** dictionary.

Loading Dialogue Scripts

Three options are available for loading dialogue scripts into the **running_dialogue** dictionary:

Direct Manual Copy/Paste

If you have a single dialogue script, you can copy and paste it directly in the **running_dialogue** dictionary.

If you have multiple dialogue scripts, and if their conversations don't share the same names, you can copy and paste all of their conversations to the dictionary. Note that the Candy Dialogue Creator allows you to import Conversations from multiple dialogues .txt files, or to export Conversations from multiple dialogues to a single .txt file.

Built-in Storage

Alternately, you can manually copy and paste dialogue scripts to custom dictionaries in your game, then either write a small function to make your game load from the correct dictionary to **running_dialogue**, or point **running_dialogue** to those dictionaries, like so:

```
scripted_dialogues = your_custom_dialogue_dictionary
```

Load from File

The last option is to simply call the `load_scripted_dialogue()` function in `Candy_Engine.gd` at runtime, as needed. This function finds the dialogue .txt file you specify, and loads a copy of that dialogue to `running_dialogue`.

Note that the function will look for dialogue files in the folder specified in the `dialogues_folder` variable in `Candy_Properties.gd`.

Starting Dialogues

To start a dialogue, call the `start_dialogue()` function in `Candy_Engine.gd`, like so:

```
candy_ui.start_dialogue(conversation, block, line)
```

The arguments represent the Conversation, Block and Line you want to start at.

For conversation and block, provide the name of the Conversation and Block as they appear in your dialogue script.

To start at the first line, which you'll likely do most of the time, write a value of 0. If you want to start at another line, either provide the line index or a Line Mark reference (see the Transitions guide for details about Line Markers).

Remember that lines are stored inside an array, so indexing starts at 0, not 1.

Ending Dialogues

Dialogues automatically end after the last line is reached and processed.

When ending dialogues, Candy DE does the following:

1. If you use a game state tracker and you configured the `change_game_state()` function in `Candy_Functions.gd`, it reverts the game state to what it was before dialogue began.
2. It clears the dialogue UI: hiding UI elements, removing portraits, VN bust images, text, etc.
3. It changes Mouse_Mode as per your settings in `Candy_Engine.gd`.
4. It calls the `x_dialogue_end()` hook function (`Candy_Functions.gd`).

If you want the engine to do anything else when dialogue ends, that's what the `x_dialogue_end()` function is for: it's empty by default, and its purpose is to let you inject additional events upon dialogue termination.

It's called as the last step of the `end_dialogue()` function in `Candy_Engine.gd`. You can move the call earlier in the function if you'd like, or add your own calls to more hook functions of your own creation. You can also modify the function and the call to pass arguments to it.